TRIP PLANNER PWA · FICHE MODULE · MEMBRE 6

Module 6 — Gestion du journal de voyage

Ce que fait le module, la logique métier qu'il porte, et ce que vous devez savoir présenter.

COLLECTION MONGODB	FONCTIONNALITÉ AVANCÉE	RÔLE TRANSVERSE
journal_entries	Générateur d'itinéraire par IA (à réaliser)	—

1. Objectif du module

Permettre aux voyageurs de garder une trace de chaque journée : entrées datées avec texte, photos, humeur, lieu de l'itinéraire et position GPS, lisibles et modifiables même sans réseau. Le module porte aussi l'envoi d'images (utilisé pour les reçus) et la future génération d'itinéraire par IA.

> Règle d'équipe : vous êtes le seul à écrire dans votre collection. Les autres modules passent par votre API (ou votre service), et toute modification d'un modèle partagé demande l'accord du groupe.

2. Modèle de données

Champ	Type	Règle / rôle
`trip`	ref trips	obligatoire
`author`	ref users	auteur de l'entrée
`place`	ref places | null	lieu de l'itinéraire (même voyage)
`title`	texte	≤ 120 caractères, optionnel
`body`	texte	1 à 10 000 caractères
`photos[]`	URLs	12 au maximum
`date`	date	jour raconté
`mood`	enum	amazing, happy, neutral, tired, sad
`lat, lng`	nombres | null	position GPS au moment de l'écriture

3. Logique métier

Règles du journal

- Tous les membres du voyage lisent toutes les entrées et peuvent en écrire.
- Seul l'**auteur** peut modifier ou supprimer son entrée (403 sinon) ; l'interface masque les boutons pour les autres.
- Le lieu rattaché doit appartenir au **même voyage** (sinon 400).
- Tri par date décroissante ; filtre possible par lieu.
- Création idempotente (id client), modification en last-write-wins, suppression idempotente.

Hors ligne (exigence du cahier des charges)

- Les entrées sont **lisibles et modifiables sans réseau** : texte, humeur, lieu et position GPS (API de géolocalisation du navigateur).

- Elles sont mises en file d'attente et envoyées à la reconnexion.
- L'ajout de **photos** nécessite une connexion (l'interface l'indique).

Envoi d'images (module uploads)

- `POST /uploads?folder=journal|receipts` : jusqu'à 6 images par envoi, 5 Mo maximum chacune, formats JPEG, PNG, WebP, GIF, AVIF.
- Stockage sur **Cloudinary** (converties en WebP) si les clés sont configurées, sinon dans le dossier `uploads/` de l'API.
- Affichage en tailles adaptées (paramètres Cloudinary de largeur et qualité automatique).

4. Fonctionnalité avancée — Générateur d'itinéraire par IA (à réaliser)

À réaliser — générateur d'itinéraire par IA (spécification à suivre) :

– un endpoint, par exemple `POST /trips/:tripId/ai-itinerary`, reçoit la destination, le nombre de jours et les centres d'intérêt ;

– l'API appelle le modèle d'IA **côté serveur** (clé dans les variables d'environnement, jamais dans le client).

La réponse doit être demandée en **JSON structuré** (jours → lieux : nom, latitude, longitude, catégorie, heure, durée, coût) puis **validée avec zod**. Les lieux invalides sont rejetés.

Après validation par l'utilisateur (aperçu), les lieux sont insérés comme de **vrais lieux** du module 3, modifiables ensuite, en passant par la logique du module 3 (règle « on n'écrit pas dans la table d'un autre membre »).

Une **limite d'appels** (rate limiting) s'applique à cet endpoint, comme exigé par la sécurité du cahier des charges.

Risque identifié : l'IA lente ou indisponible pendant la soutenance. Mitigation prévue : itinéraires déjà générés pour **deux destinations de démo**, enregistrés en base.

5. API REST (préfixe `/api/v1`, réponse `{ data, message, errors }`)

Méthode	Route	Accès	Rôle
`GET / POST`	`/trips/:tripId/journal`	membre	liste (filtre place) / nouvelle entrée
`GET / PATCH / DELETE`	`/trips/:tripId/journal/:entryId`	membre (écriture : auteur)	détail, modifier, supprimer
`POST`	`/uploads?folder=…`	membre	envoi d'images
`POST`	`/trips/:tripId/ai-itinerary`	membre	*à créer* — génération IA

6. Écrans (front Next.js)

- Onglet « Journal » : chronologie des entrées (humeur, lieu, auteur, photos)
- Formulaire : date, lieu, titre, texte, humeur, « Add my location », photos
- *À créer* : écran de génération IA (destination, durée, intérêts) avec aperçu avant insertion

7. Fichiers dont vous êtes responsable

API — trip-planner-api

- `src/modules/journal/ (journal.model, journal.routes)`

Front — trip-planner-web

- `src/app/trips/[id]/journal/page.tsx`
- `fonction uploadImages dans src/lib/api.ts`

- `src/modules/uploads/upload.routes.ts`
- `src/lib/upload.ts`

8. Liens avec les autres modules

- **Module 2** : permissions et file hors ligne.
- **Module 3** : rattachement à un lieu ; le générateur IA créera des lieux.
- **Module 4** : les reçus utilisent le même service d'envoi d'images.

9. À démontrer à la soutenance

- ☐ Écrire une entrée avec humeur, lieu, photos et position GPS.
- ☐ Hors ligne : écrire une entrée, recharger la page (elle est toujours là), puis la voir synchronisée chez un autre participant.
- ☐ Montrer qu'un autre participant ne peut pas modifier votre entrée.
- ☐ (Une fois réalisé) générer un itinéraire IA et l'insérer dans le voyage.

10. Reste à faire

- **Générateur d'itinéraire IA** (fonctionnalité avancée du module) + limite d'appels + itinéraires de secours en cache.
- Option : photos hors ligne (stocker l'image dans IndexedDB et l'envoyer à la reconnexion).
- Tests d'intégration ; requêtes Postman.

11. Questions probables du jury

Pourquoi les photos ne marchent-elles pas hors ligne ?
L'envoi d'un fichier vers Cloudinary demande le réseau ; le texte, lui, est mis en file d'attente. C'est une amélioration possible.

Comment fiabiliser la réponse de l'IA ?
En imposant un format JSON, en le validant avec zod, et en montrant un aperçu avant d'enregistrer.